

system access and prevents arbitrary memory reads/writes, making it inherently more secure for running untrusted code or small plug-ins.

Portability and deployment flexibility: Docker images are often tied to specific operating systems and processor architectures, which can complicate deployments across heterogeneous environments. Although multi-architecture Docker builds exist, they require additional configuration and image management. Wasm modules, however, compile into a platform-agnostic binary that runs identically on any Wasm runtime, regardless of OS or hardware. This makes Wasm an excellent choice for truly portable workloads in edge computing and browser-based applications.

Let us set up an environment to check which one performs better — Docker or Wasm.

Part A: Environment setup

Installing the required tools: To perform a fair and repeatable comparison, we need a common development environment with the following tools:

- Docker (with the ability to build and run containers)
- WASI SDK (to compile C programs to WebAssembly)
- Wasmtime (to execute *.wasm* binaries)
- Clang (or the Clang bundled with the WASI SDK)
- */usr/bin/time* (to measure execution time)

After setting up these tools, verify that both the Docker version and Wasmtime version return valid outputs. This confirms your environment is ready.

Follow these steps to install the tools needed for this hands-on comparison.

Docker setup: Install Docker from the official site or use the command below (for Ubuntu):

```
$sudo apt update
$sudo apt install -y docker.io
$sudo usermod -aG docker $USER
```

Log out and log back in to apply the group change. Verify the setup by running:

```
$docker --version
```

WASI SDK setup: Download and extract the WASI SDK:

```
wget https://github.com/WebAssembly/wasi-sdk/releases/download/wasi-sdk-20/wasi-sdk-20.0-linux.tar.gz

mkdir -p $HOME/wasi-sdk
tar -xzf wasi-sdk-20.0-linux.tar.gz -C $HOME/wasi-sdk
--strip-components=1
export WASI_SDK_PATH=$HOME/wasi-sdk
```

Add to *.bashrc* for persistence:

```
echo 'export WASI_SDK_PATH=$HOME/wasi-sdk' >> ~/.bashrc
```

Wasmtime setup: Install Wasmtime by following the steps below:

```
curl https://wasmtime.dev/install.sh -sSf | bash
export PATH="$HOME/.wasmtime/bin:$PATH"
```

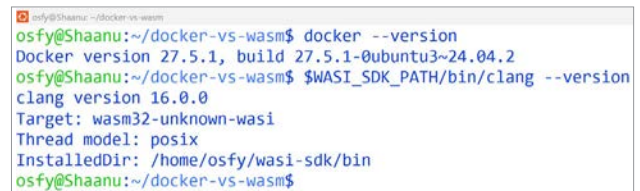
Make it permanent:

```
echo 'export PATH="$HOME/.wasmtime/bin:$PATH"' >> ~/.bashrc
```

Time utility setup: Ensure you have */usr/bin/time*:

```
sudo apt install -y time
```

The installation was successfully verified with Docker version 27.5.1 and WASI SDK (Clang) version 16.0.0 on Ubuntu 24.04, as shown in Figure 1.



```
osfy@Shaanu: ~/docker-vs-wasm
osfy@Shaanu:~/docker-vs-wasm$ docker --version
Docker version 27.5.1, build 27.5.1-0ubuntu3~24.04.2
osfy@Shaanu:~/docker-vs-wasm$ $WASI_SDK_PATH/bin/clang --version
clang version 16.0.0
Target: wasm32-unknown-wasi
Thread model: posix
InstalledDir: /home/osfy/wasi-sdk/bin
osfy@Shaanu:~/docker-vs-wasm$
```

Figure 1: Docker and Wasm installation

Part B: Sample application

For consistency, we use the same C source code to test both runtime environments. The program is a simple greeting message:

```
#include <stdio.h>

int main() {
    printf("Hello from Docker Container!\n");
    return 0;
}
```

This program is saved as *hello-d.c* and will be compiled into a Linux binary for Docker.

```
#include <stdio.h>

int main() {
    printf("Hello from Web Assembly !\n");
    return 0;
}
```

This program is saved as *hello-w.c* and will be compiled into a *.wasm* file for Wasmtime.